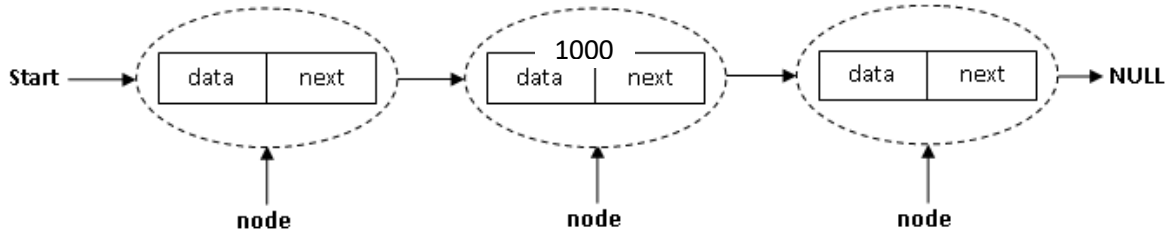
Unit3 - Linked List

Basic Terminologies of Linked List

Linked List is a **linear** data structure, in which elements are **not** stored at a contiguous location; rather they are linked using pointers.

Linked List forms a series of connected nodes, where each node stores the data and the address of the next node.



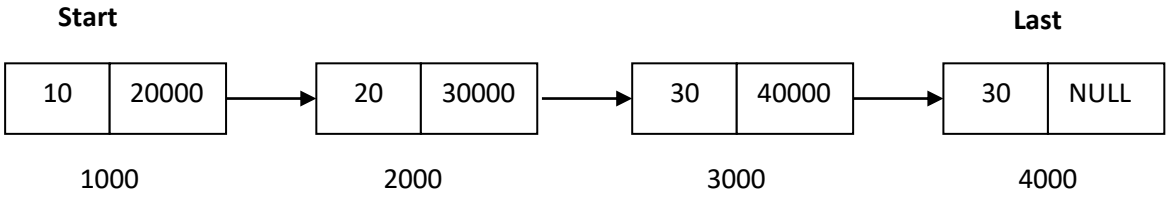
Node Structure:

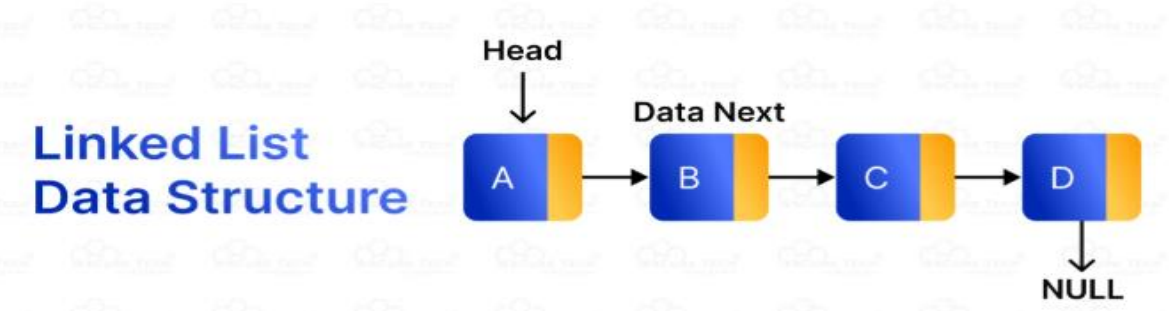
A node in a linked list typically consists of two parts:

1. **data:** It holds the actual value or data associated with the node.
2. **next Pointer:** It stores the memory address (reference) of the next node in the sequence.

Head/start: The linked list is accessed through the **start node**, which points to the first node in the list.

Tail/last: The **last node** in the list points to NULL, indicating the end of the list. This node is known as the last/tail node.





Comparison of Array and Linked List

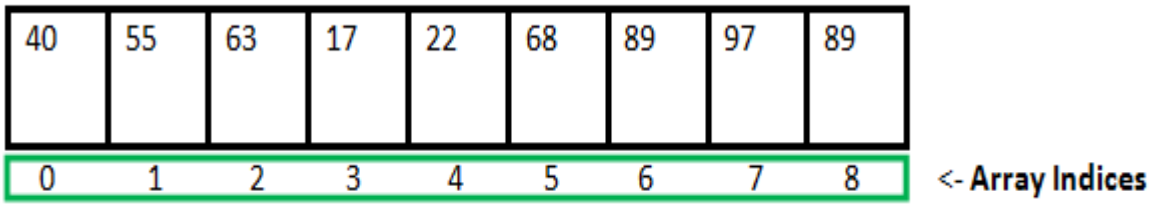
Sr. No.	ARRAY	LINKED LIST
1.	An array is a collection of data elements of same data type.	A linked list is a group of entities called a node. The node includes two segments: data and address.
2.	It stores the data elements in a contiguous memory zone.	It stores elements randomly i.e. anywhere in the memory zone.
3.	Memory size is fixed, and it is not possible to change it during the run time.	In the linked list, the placement of elements is allocated during the run time.
4.	The elements are not dependent on each other.	The data elements are dependent on each other.
5.	The memory is assigned at compile time.	The memory is assigned at run time.
6.	It is easier and faster to access the element in an array.	In a linked list, the process of accessing elements takes more time.
7.	Memory utilization is ineffective.	Memory utilization is effective.
8.	When it comes to executing any operation like insertion, deletion, array takes more time.	When it comes to executing any operation like insertion, deletion, the linked list takes less time.

Operation	Linked list	Array
Random Access	O(n)	O(1)
Insertion and deletion at beginning	O(1)	O(n)
Insertion and deletion at end	O(n) (If we maintain only head)	O(1)
Insertion and deletion at a random position	O(n)	O(n)

What is a Static Data structure?

In Static data structure the size of the structure is fixed. The content of the data structure can be modified but without changing the memory space allocated to it.

Example: Array



Array Length = 9
First Index = 0
Last Index = 8

What is Dynamic Data Structure?

In Dynamic data structure the size of the structure is not fixed and can be modified during the operations performed on it. Dynamic data structures are designed to facilitate change of data structures in the run time.

Example: Linked List

Aspect	Static Data Structure	Dynamic Data Structure
Memory allocation	Memory is allocated at compile-time	Memory is allocated at run-time
Size	Size is fixed and cannot be modified	Size can be modified during runtime
Memory utilization	Memory utilization may be inefficient	Memory utilization is efficient as memory can be reused
Access	Access time is faster as it is fixed	Access time may be slower due to indexing and pointer usage
Examples	Arrays, Stacks, Queues, Trees (with fixed size)	Lists, Trees (with variable size), Hash tables

Advantage of Static data structure:

- **Fast access time**: Static data structures offer fast access time because memory is allocated at compile-time and the size is fixed, which makes accessing elements a simple indexing operation.
- **Predictable memory usage:** Since the memory allocation is fixed at compile-time, the programmer can precisely predict how much memory will be used by the program, which is an important factor in memory-constrained environments.
- **Ease of implementation and optimization:** Static data structures may be easier to implement and optimize since the structure and size are fixed, and algorithms can be

optimized for the specific data structure, which reduces cache misses and can increase the overall performance of the program.

- **Efficient memory management:** Static data structures allow for efficient memory allocation and management. Since the size of the data structure is fixed at compile-time, memory can be allocated and released efficiently, without the need for frequent reallocations or memory copies.
- **Simplified code:** Since static data structures have a fixed size, they can simplify code by removing the need for dynamic memory allocation and associated error checking.
- **Reduced overhead:** Static data structures typically have lower overhead than dynamic data structures, since they do not require extra bookkeeping to manage memory allocation and deallocation.

Disadvantages of Static Data Structures:

- The size and structure of static data structures are fixed, which can lead to **inefficient memory usage** if the data requirements change.
- Static data structures have a **predefined size**, limiting their ability to handle large and varying amounts of data.
- If the data size is smaller than the allocated memory, the **unused memory is wasted**.
- Expanding the size of a static data structure requires complex procedures, such as creating a new, larger structure and copying the data.

Advantage of Dynamic Data Structure:

- **Flexibility:** Dynamic data structures can grow or shrink at runtime as needed, allowing them to adapt to changing data requirements. This flexibility makes them well-suited for situations where the size of the data is not known in advance or is likely to change over time.
- **Reduced memory waste:** Since dynamic data structures can resize themselves, they can help reduce memory waste. For example, if a dynamic array needs to grow, it can allocate additional memory on the heap rather than reserving a large fixed amount of memory that might not be used.
- **Improved performance for some operations:** Dynamic data structures can be more efficient than static data structures for certain operations. For example, inserting or deleting elements in the middle of a dynamic list can be faster than with a static array, since the remaining elements can be shifted over more efficiently.
- **Simplified code**: Dynamic data structures can simplify code by removing the need for manual memory management. Dynamic data structures can also reduce the complexity of code for data structures that need to be resized frequently.
- **Scalability**: Dynamic data structures can be more scalable than static data structures, as they can adapt to changing data requirements as the data grows.

Disadvantages of Dynamic Data Structures:

- Dynamic memory allocation and deallocation can incur **additional overhead** compared to static data structures.
- Dynamic data structures can be **more complex** to implement and maintain than static data structures.

- Improper handling of dynamic memory can lead **to memory leaks**, which can cause performance issues.
- The performance of dynamic data structures can be **less predictable** than static data structures, especially in real-time systems.

Types of Linked List in Data Structures

There are three primary types of linked lists:

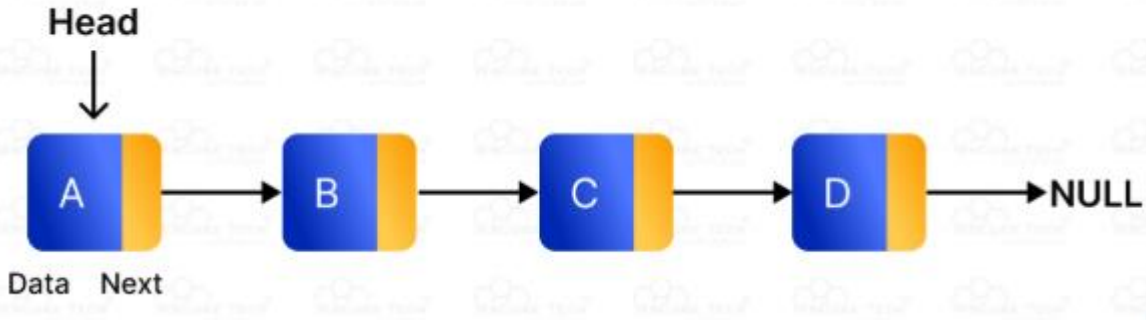
1. Singly Linked List: A singly linked list is a collection of nodes where each node contains data and a reference to the next node in the sequence. This structure allows for a simple, one-directional traversal from the head to the end of the list.

Example:

Consider managing a simple task list. Each task points to the next one until the end:

Task1 -> Task2 -> Task3 -> null

Operations like adding or removing tasks involve adjusting the 'next' reference of the preceding node, which is straightforward but only allows moving forward through the list.



2. Doubly Linked List

Doubly linked lists expand on singly linked lists by having each node contain two references: one pointing to the next node and one to the previous node. This allows nodes to be traversed in both forward and backward directions.

Example:

Imagine a playlist where you can move to the next or previous song:

null <- PrevSong <-> CurrentSong <-> NextSong -> null

This setup is particularly useful for applications like image viewers or document viewers where users may need to go forward and backward between images or pages.

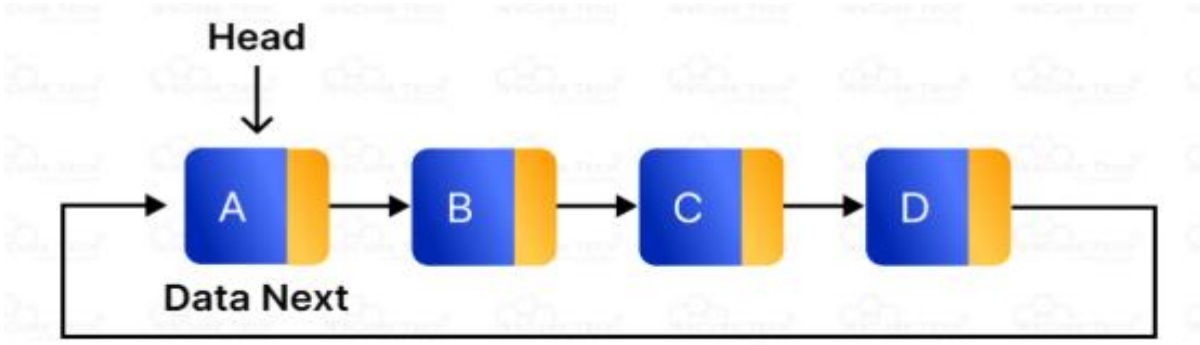
3. Circular Linked List

A **circular linked list** is a **type of data structure** where each node points to the next node, and the last node points back to the first, forming a circle.

This setup allows for an endless cycle of node traversal, which can be particularly useful in applications where the end of the list naturally reconnects to the beginning.

Example:

In multiplayer games, a circular linked list manages player turns, cycling back to the first player after the last one finishes.



Uses of Linked Lists:

Data structure linked lists offer several practical uses across different applications:

1. Dynamic Memory Allocation:

Linked lists are ideal for applications where the amount of data is unknown in advance and can change dynamically. They are extensively used in implementing memory management **algorithms**.

2. Implementing Stacks and Queues:

Linked lists provide a natural way to implement other abstract data types **like stacks and queues.**

For example, a singly linked list can serve as a stack or queue by adding or removing nodes from one end.

3. Browser History Management:

The functionality of back and forward navigation in web browsers can be effectively managed using a doubly linked list where each node points to the previous and next web pages visited.

4. Undo Functionality in Applications:

Linked lists are used in implementing undo mechanisms in software applications, where operations are stored in a list and can be reversed by traversing backwards.

5. Music Playlists and Image Viewers:

Applications that require sequential access to elements, such as media players or image viewers, often use doubly linked lists to facilitate easy navigation between items (next and previous).

6. Graphs Representation:
Adjacency lists, which are used to represent graphs, can be implemented using linked lists, where each node represents a vertex and its linked list represents the adjacent vertices.

7. Polynomial Arithmetic:
Linked lists are useful in representing and manipulating polynomials. Each node can represent a term of the polynomial, allowing for dynamic adjustments to the polynomial as terms are added or subtracted.

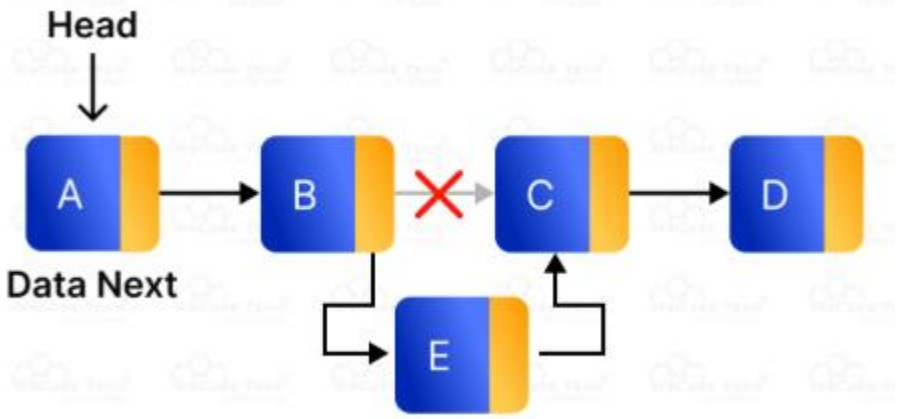
Linked List Operations:

Understanding how to perform operations on linked lists is crucial for using them effectively in programming.

1. Insertion
Insertion in a linked list includes adding a new node to the list. You can insert a node at the beginning, in the middle, or at the end of the list.

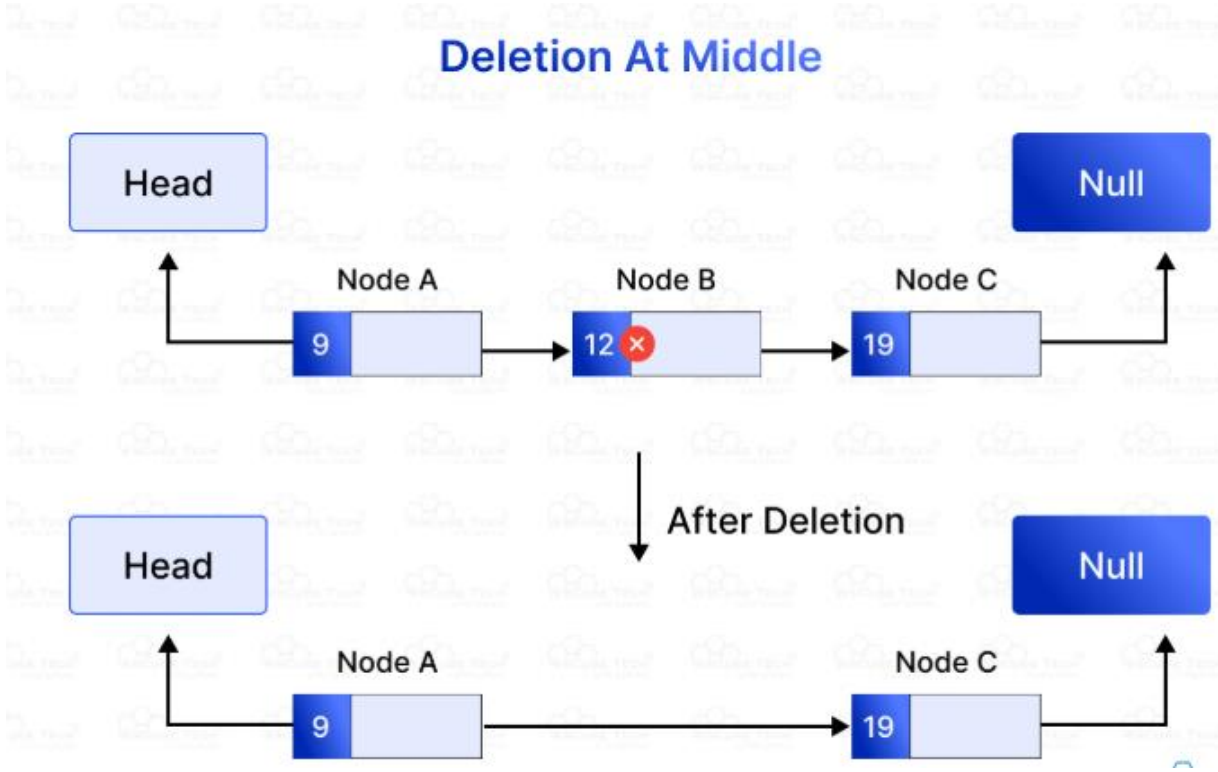
Example:
Inserting at the beginning: Imagine you have a list of daily tasks, and you suddenly need to add a new task at the top of your list. You create a new node for this task and make it the new head of the list, pointing to the previous first task.

"Read book" -> becomes new head -> "Go to market" -> "Write essay" -> null



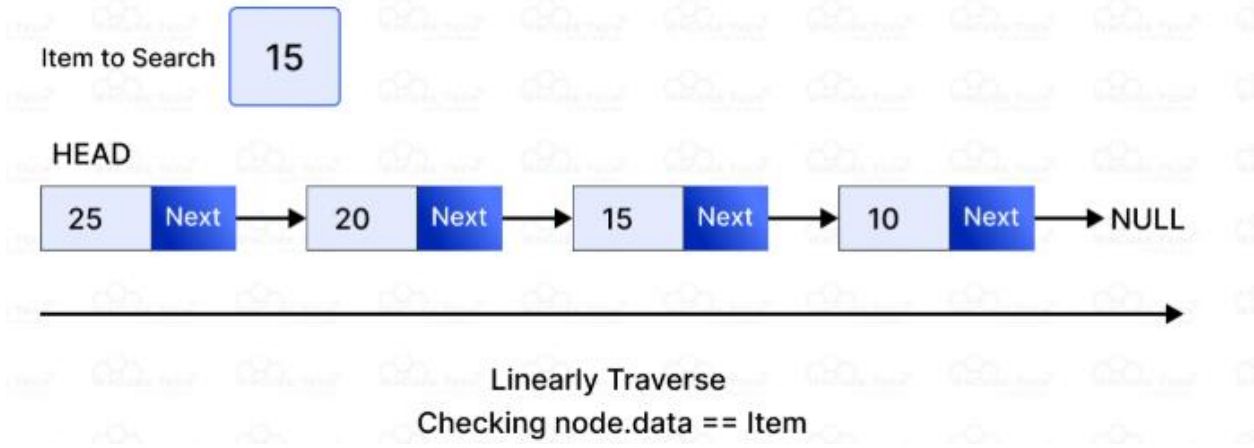
2. Deletion

Deletion removes a node from the linked list. This can also be done at the beginning, at a specific position, or at the end.



3. Search

Searching in a linked list involves traversing through the list from the head to find a node containing a specific value.



4. Traversal

Traversal means going through each node in the list from the beginning to the end to access or modify data.

5. Update

Updating involves changing the data in one of the nodes without altering the structure of the list.

Advantages of Linked Lists

- **Dynamic Sizing:** Linked lists can grow and shrink during runtime as they do not have a fixed size, unlike arrays.
- **Efficient Insertions and Deletions:** Adding or removing nodes does not require the elements to be contiguous in memory, so operations can be performed without reallocating or reorganizing the entire structure.
- **No Memory Waste:** Since there's no need to pre-allocate memory, linked lists can manage data more efficiently without wasting memory on unused space.
- **Ease of Implementation:** Can easily be implemented and manipulated, particularly when it comes to complex data structures such as stacks, queues, and other abstract data types.
- **Flexible Structure:** Nodes can easily be rearranged by changing their next pointers without the need for data movement, which is beneficial for applications that require frequent reordering of data.

Disadvantages of Linked Lists (Limitations)

- **Higher Memory Use:** Each node in a linked list requires additional memory for a pointer to the next (and possibly previous) node, which is more memory-intensive than the tight data packing in arrays.
- **Slower Access Time:** Direct access is not feasible with linked lists. To access an element at a specific index, you have to traverse the list from the head to that position, which takes more time than array access.
- **Complexity in Navigation:** Navigating a linked list can be more complex compared to navigating an array, as it requires pointer manipulation, which can be prone to errors like pointer corruption or memory leaks.
- **Extra Memory for Pointers:** The requirement for storing pointers typically means using extra memory, which can be significant in systems with a large number of elements.

Time Complexity of Linked List Operations

Operation	Worst Case Complexity	Average Case Complexity
Access	O(n)	O(n)
Search	O(n)	O(n)
Insertion		
- At the beginning	O(1)	O(1)
- At the end	O(n)	O(n)
- In the middle	O(n)	O(n)
Deletion		
- At the beginning	O(1)	O(1)
- At the end	O(n)	O(n)
- In the middle	O(n)	O(n)